Jhanvi Cars — EKS Deployment Command Reference

A step-by-step command guide to move the backend from Minikube to AWS EKS, with RDS for PostgreSQL. Practice reference — verify account IDs, regions, and resource names before running in your own AWS account.

Assumes: AWS CLI configured with credentials, an existing Docker image for the backend, and the Terraform VPC/networking groundwork already in ap-south-2 (adjust region as needed).

0. Prerequisites — install required tools

```
# AWS CLI
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip && sudo ./aws/install
aws configure   # enter Access Key, Secret Key, region (ap-south-2), output format

# eksctl (cluster creation tool)
curl --silent --location "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_
sudo mv /tmp/eksctl /usr/local/bin

# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# helm (needed for AWS Load Balancer Controller)
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# verify installs
aws --version
eksctl version
kubectl version --client
helm version
```

1. Push your backend image to Amazon ECR

EKS pulls images from a registry it can reach — Docker Hub works too, but ECR is the standard senior-level answer (private, IAM-integrated, no rate limits).

```
# Create the ECR repository
aws ecr create-repository \
  --repository-name jhanvi-cars-backend \
  --region ap-south-2

# Authenticate Docker to ECR
aws ecr get-login-password --region ap-south-2 | \
  docker login --username AWS --password-stdin \
  <ACCOUNT_ID>.dkr.ecr.ap-south-2.amazonaws.com

# Tag and push your existing image
docker tag jhanviadmin/jhanvi-cars-backend:1.0 \
  <ACCOUNT_ID>.dkr.ecr.ap-south-2.amazonaws.com/jhanvi-cars-backend:1.0

docker push <ACCOUNT_ID>.dkr.ecr.ap-south-2.amazonaws.com/jhanvi-cars-backend:1.0
```

```
# Confirm
aws ecr list-images --repository-name jhanvi-cars-backend --region ap-south-2
```

2. Create the EKS cluster

Two paths: eksctl (fast, good for practice) or Terraform (matches what you already started — recommended long-term since it's declarative and version-controlled).

2a. Quick path — eksctl

```
eksctl create cluster \
  --name jhanvi-cars-cluster \
  --region ap-south-2 \
  --vpc-private-subnets=<PRIVATE_SUBNET_IDS> \
  --vpc-public-subnets=<PUBLIC_SUBNET_IDS> \
  --nodegroup-name jhanvi-cars-nodes \
  --node-type t3.medium \
  --nodes 2 \
  --nodes-min 2 \
  --nodes-max 4 \
  --managed

# This takes 15-20 minutes. eksctl auto-updates your kubeconfig on completion.
```

2b. Terraform path (uses your existing VPC module)

```
# In your terraform/ directory, add an eks.tf using the terraform-aws-modules/eks module,
# referencing the VPC/subnets you already created.

terraform init
terraform plan -out=eks.tfplan
terraform apply eks.tfplan

# Update kubeconfig once the cluster exists
aws eks update-kubeconfig --region ap-south-2 --name jhanvi-cars-cluster
```

3. Verify cluster access

```
kubectl get nodes -o wide
kubectl get svc
kubectl cluster-info
aws eks describe-cluster --name jhanvi-cars-cluster --region ap-south-2 --query "cluster.status"
```

4. Create namespace and secrets

```
kubectl create namespace jhanvi-cars

kubectl create secret generic jhanvi-cars-secrets \
  --namespace jhanvi-cars \
  --from-literal=JWT_SECRET='<your-jwt-secret>' \
  --from-literal=DB_PASSWORD='<your-db-password>' \
  --from-literal=GOOGLE_ROUTES_API_KEY='<your-api-key>'

# Verify (keys only, not values)
kubectl describe secret jhanvi-cars-secrets -n jhanvi-cars
```

5. Provision PostgreSQL on RDS (replacing the in-cluster Postgres pod)

Production direction from your architecture doc — managed Postgres in private subnets rather than a pod.

```
aws rds create-db-instance \
  --db-instance-identifier jhanvi-cars-db \
  --db-instance-class db.t3.micro \
  --engine postgres \
  --engine-version 16 \
  --master-username postgres \
  --master-user-password '<your-db-password>' \
  --allocated-storage 20 \
  --db-name cabbooking \
  --vpc-security-group-ids <RDS_SECURITY_GROUP_ID> \
  --db-subnet-group-name <YOUR_DB_SUBNET_GROUP> \
  --no-publicly-accessible \
  --region ap-south-2

# Check status until "available"
aws rds describe-db-instances \
  --db-instance-identifier jhanvi-cars-db \
  --query "DBInstances[0].DBInstanceStatus" --region ap-south-2

# Get the endpoint to use as DB_HOST
aws rds describe-db-instances \
  --db-instance-identifier jhanvi-cars-db \
  --query "DBInstances[0].Endpoint.Address" --region ap-south-2
```

Alternative: manage this RDS instance via Terraform instead of the CLI, so it lives alongside your VPC/EKS state.

6. Update your Deployment manifest for EKS

Two changes from your Minikube manifest: image path now points to ECR, and DB_HOST points to the RDS endpoint instead of the in-cluster 'postgres' service.

```
# In your Deployment YAML:
#   image: <ACCOUNT_ID>.dkr.ecr.ap-south-2.amazonaws.com/jhanvi-cars-backend:1.0
#   env:
#     - name: DB_HOST
#       value: "<rds-endpoint-from-step-5>"
#     - name: JWT_SECRET
#       valueFrom:
#         secretKeyRef: { name: jhanvi-cars-secrets, key: JWT_SECRET }
#     - name: DB_PASSWORD
#       valueFrom:
#         secretKeyRef: { name: jhanvi-cars-secrets, key: DB_PASSWORD }

kubectl apply -f k8s/backend-deployment.yaml -n jhanvi-cars
kubectl apply -f k8s/backend-service.yaml -n jhanvi-cars

kubectl rollout status deployment/jhanvi-cars-backend -n jhanvi-cars
kubectl get pods -n jhanvi-cars -o wide
```

7. Expose the app: install AWS Load Balancer Controller + Ingress

This replaces NodePort — the production-correct way to expose the backend externally with TLS termination.

```
# IAM policy + service account for the controller
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/ins

aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json

eksctl create iamserviceaccount \
  --cluster jhanvi-cars-cluster \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --attach-policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/AWSLoadBalancerControllerIAMPolicy \
  --override-existing-serviceaccounts \
  --approve

# Install the controller via Helm
helm repo add eks https://aws.github.io/eks-charts
helm repo update
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=jhanvi-cars-cluster \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller

# Verify controller is running
kubectl get deployment -n kube-system aws-load-balancer-controller

# Apply your Ingress resource (annotated for ALB)
kubectl apply -f k8s/ingress.yaml -n jhanvi-cars

# Get the ALB URL once provisioned
kubectl get ingress -n jhanvi-cars
```

8. End-to-end verification

```
kubectl get all -n jhanvi-cars
kubectl get pods -n jhanvi-cars -o wide
kubectl logs -l app=jhanvi-cars-backend -n jhanvi-cars --tail=50
kubectl get ingress -n jhanvi-cars -o wide

# Hit the health endpoint through the ALB
curl http://<ALB-DNS-NAME>/health

# Check HPA / scaling if configured
kubectl get hpa -n jhanvi-cars
kubectl top pods -n jhanvi-cars   # requires metrics-server addon on EKS
eksctl utils install-vpc-controllers... # (metrics-server via: kubectl apply -f https://github.com/k
```

9. Wire GitHub Actions to deploy to EKS

Extends your existing pipeline (build → push) with a deploy stage, since your self-hosted runner is already set up.

```
# Add AWS credentials as GitHub repo secrets: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY
```

```
# In your workflow's deploy job:
aws eks update-kubeconfig --region ap-south-2 --name jhanvi-cars-cluster
kubectl set image deployment/jhanvi-cars-backend \
  jhanvi-cars-backend=<ACCOUNT_ID>.dkr.ecr.ap-south-2.amazonaws.com/jhanvi-cars-backend:${{ github.s
  -n jhanvi-cars
kubectl rollout status deployment/jhanvi-cars-backend -n jhanvi-cars
```

10. Tear down (avoid ongoing AWS charges)

EKS control plane + NAT gateways + RDS all bill continuously. Tear down after each practice session unless you're keeping it running deliberately.

```
# Delete Ingress/ALB first (avoids an orphaned load balancer)
kubectl delete ingress --all -n jhanvi-cars

# Delete the Load Balancer Controller
helm uninstall aws-load-balancer-controller -n kube-system

# Delete RDS instance
aws rds delete-db-instance \
  --db-instance-identifier jhanvi-cars-db \
  --skip-final-snapshot \
  --region ap-south-2

# Delete the EKS cluster (eksctl path)
eksctl delete cluster --name jhanvi-cars-cluster --region ap-south-2

# OR, Terraform path
terraform destroy

# Delete ECR images/repo if no longer needed
aws ecr delete-repository --repository-name jhanvi-cars-backend --force --region ap-south-2
```

Cost checklist before you walk away: EKS control plane (~$0.10/hr), NAT gateway(s), RDS instance, ALB, EBS volumes for nodes, data transfer. Running eksctl delete cluster and rds delete-db-instance covers the big-ticket items.